

Predicting Movie Genres Based on Plot Summaries

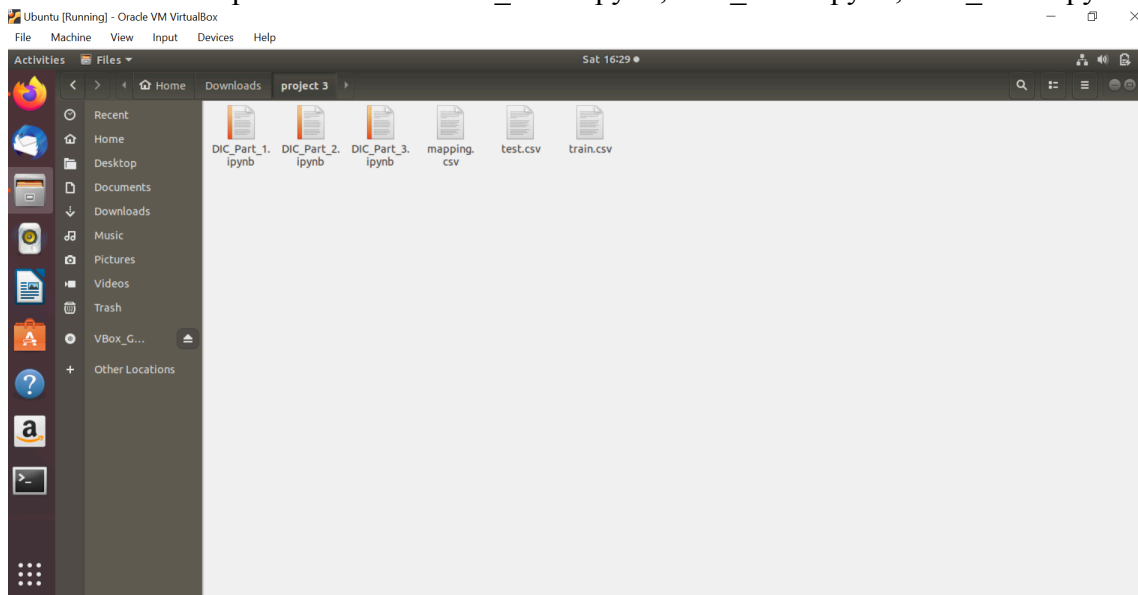
Abstract

This project depicts the development and simulation of predicting movie genres based on plot summaries by using Apache Spark. We have been given 3 datasets, train, test and mapping. The task of predicting the genre is essentially a multi-label classification problem. A movie can have multiple genres associated with it. So, our model should be able to predict all the genres associated with the movie.

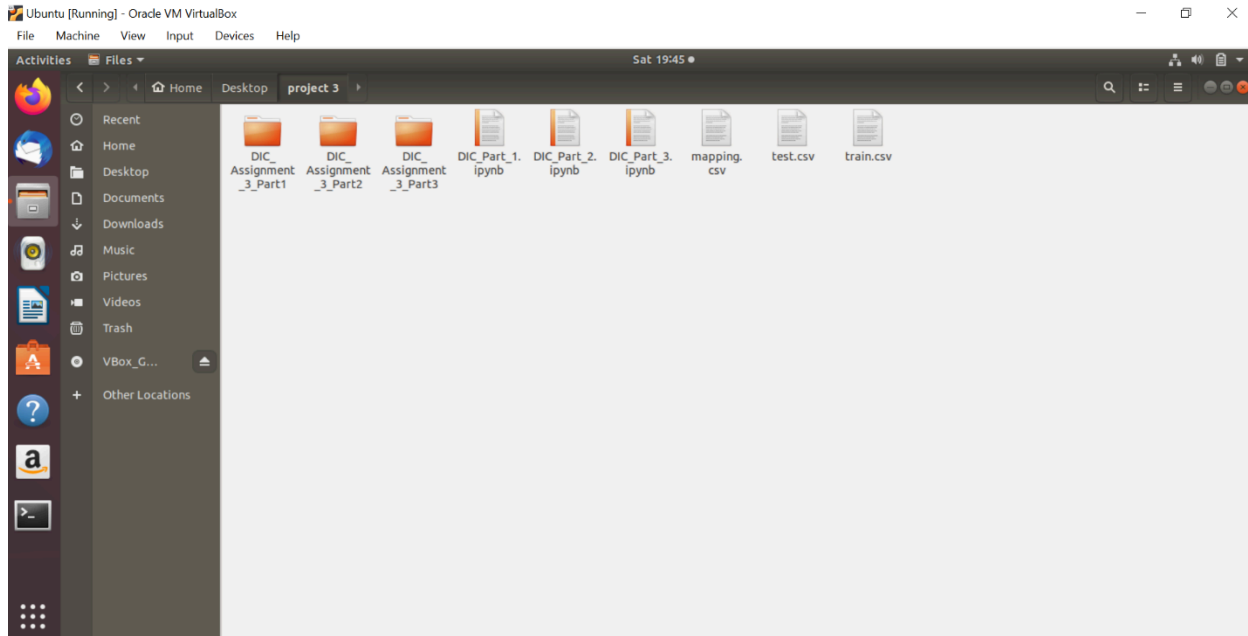


Project Configuration:

1. Create a folder project 3 and place the train.csv, test.csv and mapping.csv in the same folder. Also put all the three DIC_Part1.ipynb, DIC_Part2.ipynb, DIC_Part3.ipynb files in the same folder



Run the files that will save three prediction csv in the same folder.



◆ Dataset Definition

The train dataset has been provided with 31108 records with plot summaries. The dataset is consisting of 4 columns, namely movie_id, movie_name, plot, genre.

```
train.head()
```

	movie_id	movie_name	plot	genre
0	23890098	Taxi Blues	Shlykov, a hard-working taxi driver and Lyosha...	['World cinema', 'Drama']
1	31186339	The Hunger Games	The nation of Panem consists of a wealthy Capi...	['Action/Adventure', 'Action', 'Science Fictio...]
2	20663735	Narasimham	Poovalli Induchoodan is sentenced for six yea...	['Musical', 'Action', 'Drama']
3	2231378	The Lemon Drop Kid	The Lemon Drop Kid , a New York City swindler,...	['Comedy']
4	595909	A Cry in the Dark	Seventh-day Adventist Church pastor Michael Ch...	['Crime Fiction', 'World cinema', 'Drama']

The mapping of the genre to the string index is given by the mapping dataset as below:

```
mapping.head()
```

	genre_id	genre
0	NaN	0
1	0.0	Drama
2	1.0	Comedy
3	2.0	Romance Film
4	3.0	Thriller

We need to predict the genre for the movie plots given in the test dataset as below:

```
test.head()
```

	movie_id	movie_name	plot
0	1335380	Exodus	The film is based on the events that happened ...
1	29062594	A la salida nos vemos	A group of teenagers at Catholic boarding scho...
2	9252321	Come Back, Africa	This story of a Zulu family is a composite sto...
3	13455076	A Merry Mixup	The Stooges play three sets of identical tripl...
4	24165951	Getting Even	A soldier-of-fortune steals some Russian nerve...

As we are doing this project in pyspark, so converted all of the above-mentioned datasets into pyspark dataframe.

❖ **Data Preprocessing:**

After all the data were collected, following preprocessing was done on it using 'regex_replace' from pyspark library.

- Any/all use of http and HTML tags were removed.
- Punctuation, Numbers and special characters were removed.

❖ **Machine Learning pipeline**

Tokenization was done on cleaned data using Tokenizer class to get individual terms from the plots and all the stop words which were irrelevant to the context were removed using 'StopWordsRemover' from pyspark.ml.feature library.



Text to Features Conversions:

We need to extract features from the cleaned version of the movie plots data. For this purpose we have used 3 different approaches in 3 parts of the assignment.

1. Part 1: Basic Model

For the 1st part, we need to create a term-document matrix from the plots and use these as feature vectors for the machine learning model. We have used HashingTF, which takes sets of terms and converts those sets into fixed-length feature vectors. The terms are mapped to indices using a Hash Function. The hash function used is MurmurHash 3. The term frequencies are computed with respect to the mapped indices. We have used the 100 most frequent words in the data as our features. Same technique has been used for train and test data to extract the features.

The prepared **feature vectors for train** is as below:

movie_id	movie_name	plot	genre	words	filtered	rawFeatures
23890098	Taxi Blues	shlykov a hardwor...	['World cinema', ...]	[shlykov, a, hard...	[shlykov, hardwor...	(100,[13,19,26,31...
31186339	The Hunger Games	the nation of pan...	['Action/Adventur...	[the, nation, of,...	[nation, panem, c...	(100,[0,1,3,4,5,6...
20663735	Narasimham	poovalli induchoo...	['Musical', 'Acti...	[poovalli, induch...	[poovalli, induch...	(100,[0,1,2,3,4,5...
2231378	The Lemon Drop Kid	the lemon drop ki...	['Comedy']	[the, lemon, drop...	[lemon, drop, kid...	(100,[0,1,3,4,5,6...
595909	A Cry in the Dark	seventhday advent...	['Crime Fiction',...]	[seventhday, adve...	[seventhday, adve...	(100,[0,1,2,3,4,5...

The prepared **feature vectors for test** is as below:

movie_id	movie_name	plot	words	filtered	rawFeatures
1335380	Exodus	the film is based...	[the, film, is, b...	[film, based, eve...	(100,[0,1,3,4,5,6...
29062594	A la salida nos v...	a group of teenag...	[a, group, of, te...	[group, teenagers...	(100,[1,3,6,13,14...
9252321	Come Back, Africa	this story of a z...	[this, story, of,...	[story, zulu, fam...	(100,[1,2,3,4,5,6...
13455076	A Merry Mixup	the stooges play ...	[the, stooges, pl...	[stooges, play, t...	(100,[2,6,8,9,12,...
24165951	Getting Even	a soldieroffortun...	[a, soldieroffort...	[soldieroffortune...	(100,[6,8,10,19,3...

2. Part 2: Using TF-IDF

For the 2nd part, we implemented Term Frequency-Inverse Document Frequency (TF-IDF) based feature engineering technique. TF picks the most frequently occurring terms. We also want a measure of how unique a word is i.e. how infrequently the word occurs across all documents (inverse document frequency or idf). So, the product of tf & idf (TF-IDF) of a word gives a product of how frequent this word is in the document multiplied by how unique the word is with respect to the entire corpus of documents. Words in the document with a high tf-idf score occur frequently in the document and provide the most information about that specific document.

We have used **HashingTF-IDF** and used the 100 most frequent words in the data as our features. Same technique has been used for train and test data to extract the features.

The prepared **feature vectors for train** is as below:

movie_id	movie_name	plot	genre	words	filtered	rawFeatures	features
23890098	Taxi Blues	shlykov a hardwor...	['World cinema', ...]	[shlykov, a, hard...	[shlykov, hardwor...	(100,[13,19,26,31...	(100,[13,19,26,31...
31186339	The Hunger Games	the nation of pan...	['Action/Adventur...	[the, nation, of,...	[nation, panem, c...	(100,[0,1,3,4,5,6...	(100,[0,1,3,4,5,6...
20663735	Narasimham	poovalli induchoo...	['Musical', 'Acti...	[poovalli, induch...	[poovalli, induch...	(100,[0,1,2,3,4,5...	(100,[0,1,2,3,4,5...
2231378	The Lemon Drop Kid	the lemon drop ki...	['Comedy']	[the, lemon, drop...	[lemon, drop, kid...	(100,[0,1,3,4,5,6...	(100,[0,1,3,4,5,6...
595909	A Cry in the Dark	seventhday advent...	['Crime Fiction',...]	[seventhday, adve...	[seventhday, adve...	(100,[0,1,2,3,4,5...	(100,[0,1,2,3,4,5...

The prepared **feature vectors for test** is as below:

movie_id	movie_name	plot	words	filtered	rawFeatures	features
1335380	Exodus	the film is based...	[the, film, is, b...	[film, based, eve...	(100,[0,1,3,4,5,6...	(100,[0,1,3,4,5,6...
29062594	A la salida nos v...	a group of teenag...	[a, group, of, te...	[group, teenagers...	(100,[1,3,6,13,14...	(100,[1,3,6,13,14...
9252321	Come Back, Africa	this story of a z...	[this, story, of,...	[story, zulu, fam...	(100,[1,2,3,4,5,6...	(100,[1,2,3,4,5,6...
13455076	A Merry Mixup	the stooges play ...	[the, stooges, pl...	[stooges, play, t...	(100,[2,6,8,9,12...	(100,[2,6,8,9,12...
24165951	Getting Even	a soldieroffortun...	[a, soldieroffort...	[soldieroffortune...	(100,[6,8,10,19,3...	(100,[6,8,10,19,3...

3. Part 3: Custom Feature Engineering

In this part we have implemented Word2Vec method. The Word2Vec model transforms each document into a vector using the average of all words in the document. We have chosen vectorSize as 100. Same method is used for train and test dataset.

The prepared **feature vectors for train** is as below:

movie_id	movie_name	plot	genre	words	filtered	rawFeatures
23890098	Taxi Blues	shlykov a hardwor...	['World cinema', ...]	[shlykov, a, hard...	[shlykov, hardwor...	[-0.0028166216022...
31186339	The Hunger Games	the nation of pan...	['Action/Adventur...	[the, nation, of,...	[nation, panem, c...	[-0.0156753159511...
20663735	Narasimham	poovalli induchoo...	['Musical', 'Acti...	[poovalli, induch...	[poovalli, induch...	[-0.0275993847485...
2231378	The Lemon Drop Kid	the lemon drop ki...	['Comedy']	[the, lemon, drop...	[lemon, drop, kid...	[-0.0134173090509...
595909	A Cry in the Dark	seventhday advent...	['Crime Fiction',...]	[seventhday, adve...	[seventhday, adve...	[-0.0320682891145...

The prepared **feature vectors for test** is as below:

movie_id	movie_name	plot	words	filtered	rawFeatures
1335380	Exodus	the film is based...	[the, film, is, b...	[film, based, eve...	[0.00718820147605...
29062594	A la salida nos v...	a group of teenag...	[a, group, of, te...	[group, teenagers...	[7.99817498773336...
9252321	Come Back, Africa	this story of a z...	[this, story, of,...	[story, zulu, fam...	[0.00575758043159...
13455076	A Merry Mixup	the stooges play ...	[the, stooges, pl...	[stooges, play, t...	[0.00572593349544...
24165951	Getting Even	a soldieroffortun...	[a, soldieroffort...	[soldieroffortune...	[0.00789679026541...



Label Processing:

We have been given 20 unique labels and the corresponding index of the same. we will treat this multi-label classification problem as a Binary Relevance problem. Hence, we will now one hot encode the target variable.

After one-hot encoding the label column is looking like below:

movie_id	movie_name	plot	labels	rawFeatures
23890098	Taxi Blues	shlykov a hardwor...	1 0 0 0 0 1 0 0 0...	(100,[13,19,26,31...
31186339	The Hunger Games	the nation of pan...	1 0 0 0 1 0 0 0 0...	(100,[0,1,3,4,5,6...
20663735	Narasimham	poovalli induchoo...	1 0 0 0 1 0 0 0 0...	(100,[0,1,2,3,4,5...
2231378	The Lemon Drop Kid	the lemon drop ki...	0 1 0 0 0 0 0 0 0...	(100,[0,1,3,4,5,6...
595909	A Cry in the Dark	seventhday advent...	1 0 0 0 0 1 1 0 0...	(100,[0,1,2,3,4,5...
5272176	End Game	the president is ...	1 0 0 1 1 0 0 0 0...	(100,[0,1,2,3,4,7...
1952976	Dark Water	plot the film ope...	1 0 0 1 0 0 0 1 0...	(100,[1,3,4,5,6,7...
24225279	Sing	the story begins ...	1 0 0 0 0 0 0 0 0...	(100,[0,1,3,4,5,6...
2462689	Meet John Doe	infuriated at bei...	1 1 1 0 0 0 0 0 1...	(100,[1,2,3,4,5,6...
20532852	Destination Meatball	a line of people ...	0 0 0 0 0 0 0 0 0...	(100,[0,1,5,8,9,1...
15401493	Husband for Hire	lola attempts to...	0 1 0 0 0 0 0 0 0...	(100,[0,3,4,5,6,7...
18188932	Up and Down	milan and goran a...	1 1 0 0 0 1 1 0 0...	(100,[8,12,17,20,...
2940516	Ghost In The Noon...	bumbling pirate c...	0 1 0 0 0 0 0 0 0...	(100,[1,3,10,14,1...

Now, we need to split the label column into 20 different columns for each category of the movie.

movie_id	labels	label0	label1	label2	label3	label4	label5	label6	label7	label8	label9	label10	label11	label12	label13	label14	label15
23890098	1 0 0 0 0 1 0 0 0...	1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
31186339	1 0 0 0 1 0 0 0 0...	1	0	0	0	1	0	0	0	0	0	1	1	0	0	0	0
20663735	1 0 0 0 1 0 0 0 0...	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
2231378	0 1 0 0 0 0 0 0 0...	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
595909	1 0 0 0 0 1 1 0 0...	1	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0
5272176	1 0 0 1 1 0 0 0 0...	1	0	0	1	1	0	0	0	0	0	1	1	0	0	0	0
1952976	1 0 0 1 0 0 0 1 0...	1	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0
24225279	1 0 0 0 0 0 0 0 0...	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2462689	1 1 1 0 0 0 0 0 1...	1	1	1	0	0	0	0	0	1	0	0	0	0	0	0	0
20532852	0 0 0 0 0 0 0 0 0...	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	1
15401493	0 1 0 0 0 0 0 0 0...	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Next, we have combined feature vector with splitted label columns on the basis of movie id. So that we can pass this to the machine learning model classifier



Model Training & Evaluation:

The next step is to put the features received from previous step into the classifier model. Here we have used RandomForest model. As our data is imbalanced, hence we have used undersampling to have better accuracy. As we need to predict for all 20 labels, we iterated over 20 times to predict the genres for each plot.

movie_id	prediction0	prediction1	prediction2	prediction3	prediction4	prediction5	prediction6	prediction7	prediction8	prediction9	prediction10	prediction11	prediction12	prediction13	prediction14	prediction15	prediction16	prediction17	prediction18	prediction19
62693	1.0	1.0	1.0	0.0	0.0	1.0	0.0	1.0	1.0	1.0	0.0									
296252	0.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	0.0	0.0	1.0									
1356971	1.0	1.0	0.0	0.0	0.0	1.0	0.0	0.0	1.0	1.0	1.0									
1428872	1.0	1.0	1.0	1.0	1.0	1.0	0.0	1.0	1.0	0.0	1.0									
1582173	1.0	0.0	1.0	0.0	0.0	1.0	0.0	1.0	1.0	1.0	0.0									

All 20 predictions have been merged into a single column and saved into a csv as follows:

A	B	C	D	
movie_id	predictions			
62693	1	1	0	0
296252	0	1	1	1
1356971	1	1	1	0
1428872	1	1	1	1
1582173	1	0	1	0
1595142	1	1	1	0
1600825	0	1	1	1
1681132	0	1	1	1
2268290	0	1	0	1
3327607	0	0	1	0
3569957	1	1	1	0
4413498	0	1	1	1
4635580	0	0	0	1
4950989	1	1	0	1
5565692	0	0	0	1
7003785	1	0	1	0
8269287	0	1	1	1
9560197	0	1	1	1
10731337	0	0	0	0



Result:

For all of the three parts generated prediction csv file has been uploaded into Kaggle to check the F1 score.

1. Part 1: Basic Model

Attached is the predictions from part 1:



We have pushed this csv file to Kaggle to get the F1 score

Step 1

Upload submission file

✓ part1.csv (377.02 kB)

File Format

We expect the solution file to have 7777 prediction rows.
This file should have a header row. Please see sample
submission file on the [data page](#).

Number of Predictions

This file should have 7777 prediction rows.
This file should have a header row. Please see sample
submission file on the [data page](#).

Part1 - final

C

Movie Genre Prediction

An in class Kaggle competition to introduce kaggle as a platform

81 teams · 4 hours to go

[Overview](#)
[Data](#)
[Notebooks](#)
[Discussion](#)
[Leaderboard](#)
[Rules](#)
[Team](#)
[My Submissions](#)
[Submit Predictions](#)

Your most recent submission

Name	Submitted	Wait time	Execution time	Score
part1.csv	just now	0 seconds	0 seconds	1.00000

Complete

[Jump to your position on the leaderboard](#) ▾

Make a submission for [Chicken Crispy](#)

We have received the F1 score as 1.

2. Part 2: Using TF-IDF

Attached is the predictions from part 2:

[Make Submission](#)

Make a submission for [Chicken Crispy](#)

We have received the F1 score as 1.

3. Part 3: Custom Feature Engineering

Attached is the predictions from part 3:



Conclusion:

Acknowledgments:

necessary concepts related to Predictive Analytics, Apache **Spark** and helping in this project throughout. Also, we would like to express our sincere thanks and appreciation to Kyung Won Lee, Michael Long, Lawzeem, Chunwei Ma, Rui Li, Vinooth Rao Kulkarni to support us at every step where we find any difficulties.



References:

<https://towardsdatascience.com/countvectorizer-hashingtf-e66f169e2d4e>

<https://spark.apache.org/docs/latest/ml-features#tokenizer>

<https://medium.com/towards-artificial-intelligence/multi-class-text-classification-using-pyspark-mllib-doc2vec-dbfce5b39f2>

<https://www.analyticsvidhya.com/blog/2019/04/predicting-movie-genres-nlp-multi-label-classification/>

<https://towardsdatascience.com/multi-class-text-classification-with-pyspark-7d78d022ed35>

<https://datascienceplus.com/machine-learning-with-text-in-pyspark-part-1/>

<https://medium.com/@junwan01/oversampling-and-undersampling-with-pyspark-5dbc25cdf253>